

LAB ASSIGNMENT-10.3

2303A51337

BATCH-10

TASK-1:

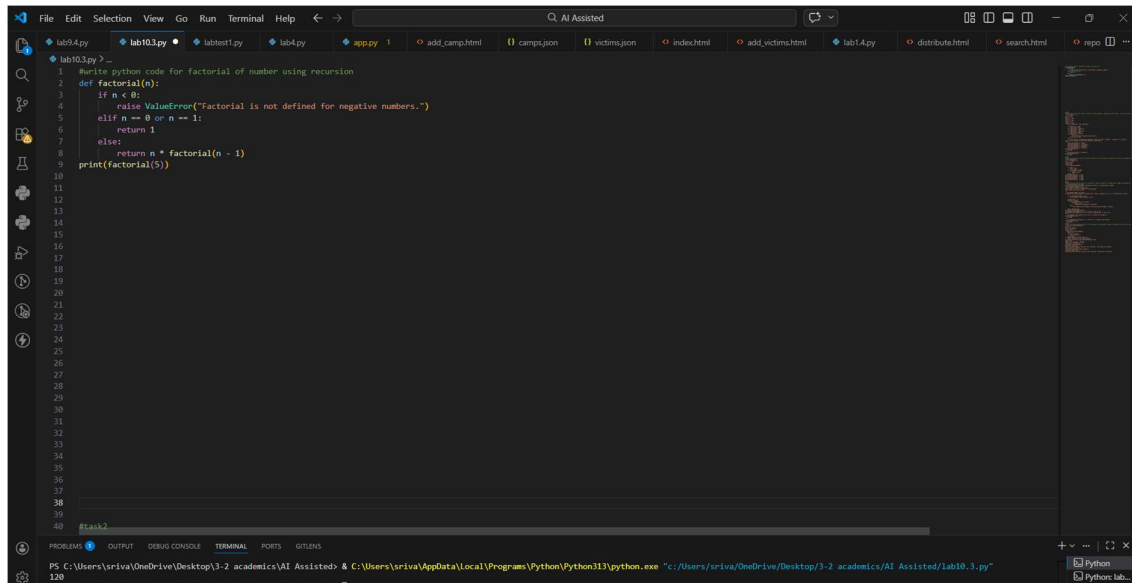
PROMPT:

write python program for factorial of number using recursion.

CODE:

```
def factorial(n):  
    if n < 0:  
        raise ValueError("Factorial is not defined for negative numbers.")  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5))
```

OUTPUT:

A screenshot of a code editor window with a dark theme. The editor shows a Python file named 'lab10.3.py' containing the factorial function code. The code is as follows:

```
1 #write python code for factorial of number using recursion  
2 def factorial(n):  
3     if n < 0:  
4         raise ValueError("Factorial is not defined for negative numbers.")  
5     elif n == 0 or n == 1:  
6         return 1  
7     else:  
8         return n * factorial(n - 1)  
9  
10 print(factorial(5))  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40
```


The output of the program is shown in the bottom right corner of the editor, displaying the number 120. The terminal at the bottom shows the command to run the file: `PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted> C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted\lab10.3.py`.

Analysis:

The function calculates factorial using recursion. If $n < 0$, it raises an error. If $n = 0$ or 1 , it returns 1 (base case). Otherwise, it returns $n \times \text{factorial}(n-1)$. $\text{factorial}(5)$ outputs 120 .

TASK-2

PROMPT:

Rewrite improve the given Python function by fixing readability, adding descriptive names, a docstring, division-by-zero handling, and input validation. Compare the original and improved versions and test with valid and invalid inputs.

```
'''def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
    elif c == "mul":  
        return a * b  
    elif c == "div":'''
```

CODE:

```
def calculate(num1, num2, operation):  
    if operation == "add":  
        return num1 + num2  
    elif operation == "subtract":  
        return num1 - num2  
    elif operation == "multiply":  
        return num1 * num2  
    elif operation == "divide":  
        if num2 == 0:  
            raise ValueError("Cannot divide by zero.")  
        return num1 / num2  
    else:  
        raise ValueError("Unsupported operation. Please use 'add', 'subtract', 'multiply', or  
'divide'.")
```

Testing the improved function with valid and invalid inputs

try:

```
print(calculate(10, 5, "add"))  
print(calculate(10, 5, "subtract"))  
print(calculate(10, 5, "multiply"))  
print(calculate(10, 5, "divide"))  
print(calculate(10, 0, "divide"))
```

except ValueError as e:

```
print(e)
```

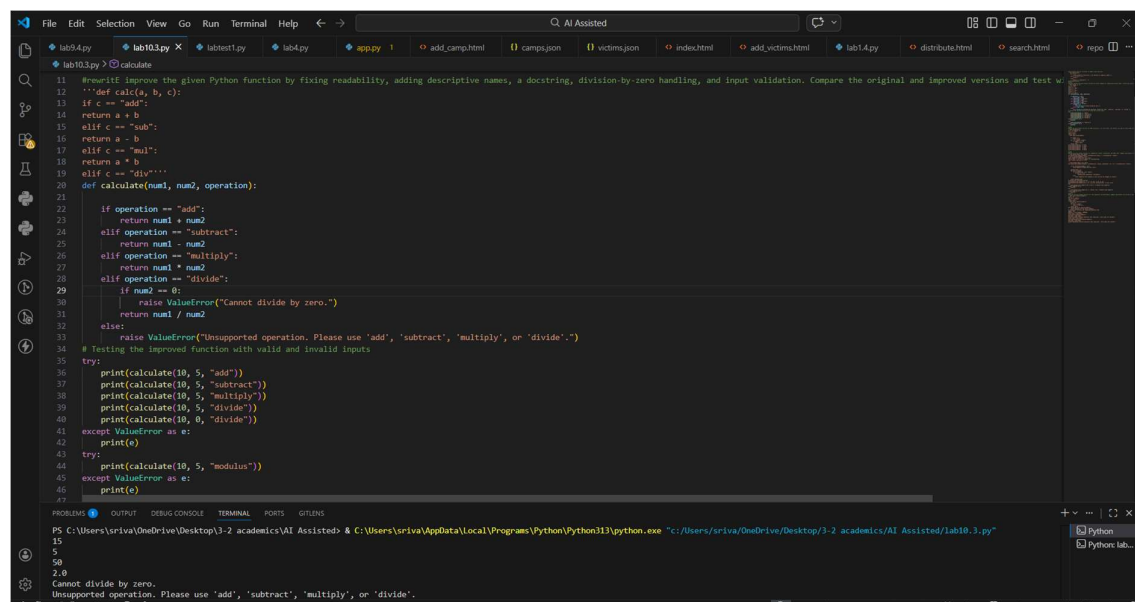
try:

```
print(calculate(10, 5, "modulus"))
```

except ValueError as e:

```
print(e)
```

OUTPUT:



The screenshot shows a VS Code editor with a Python file named 'lab10.3.py'. The code defines a function 'calculate' that takes three arguments: 'num1', 'num2', and 'operation'. It uses a series of 'if' and 'elif' statements to perform arithmetic operations: addition, subtraction, multiplication, and division. It includes error handling for division by zero (raising a 'ValueError' with the message 'Cannot divide by zero.') and for unsupported operations (raising a 'ValueError' with the message 'Unsupported operation. Please use 'add', 'subtract', 'multiply', or 'divide'.'). The code also includes a 'try' block that tests the function with various inputs, including valid ones and invalid ones like division by zero and unsupported operations. The output of the script is shown in the terminal at the bottom, which displays the results of the calculations and the error messages for the invalid inputs.

```
11 # Rewrite improve the given Python function by fixing readability, adding descriptive names, a docstring, division-by-zero handling, and input validation. Compare the original and improved versions and test w  
12 """def calc(a, b, c):  
13     if c == "add":  
14         return a + b  
15     elif c == "sub":  
16         return a - b  
17     elif c == "mul":  
18         return a * b  
19     elif c == "div":  
20         return a / b  
21  
22 def calculate(num1, num2, operation):  
23  
24     if operation == "add":  
25         return num1 + num2  
26     elif operation == "subtract":  
27         return num1 - num2  
28     elif operation == "multiply":  
29         return num1 * num2  
30     elif operation == "divide":  
31         if num2 == 0:  
32             raise ValueError("Cannot divide by zero.")  
33         return num1 / num2  
34     else:  
35         raise ValueError("Unsupported operation. Please use 'add', 'subtract', 'multiply', or 'divide'.")  
36  
37 # Testing the improved function with valid and invalid inputs  
38 try:  
39     print(calculate(10, 5, "add"))  
40     print(calculate(10, 5, "subtract"))  
41     print(calculate(10, 5, "multiply"))  
42     print(calculate(10, 5, "divide"))  
43     print(calculate(10, 0, "divide"))  
44 except ValueError as e:  
45     print(e)  
46  
47 try:  
48     print(calculate(10, 5, "modulus"))  
49 except ValueError as e:  
50     print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS
PS C:\Users\sriya\OneDrive\Desktop\3-2 academics\AI Assisted> & C:\Users\sriya\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\sriya\OneDrive\Desktop\3-2 academics\AI Assisted\lab10.3.py"
5
5
50
2.0
Cannot divide by zero.
Unsupported operation. Please use 'add', 'subtract', 'multiply', or 'divide'.

Analysis:

The function `calculate(num1, num2, operation)` performs arithmetic operations based on the given operation. It supports add, subtract, multiply, and divide. If division by zero is attempted, it raises a `ValueError`.

If an unsupported operation (like "modulus") is given, it also raises a ValueError. The try-except blocks catch these errors and print the error message instead of crashing the program.

TASK-3:

PROMPT:

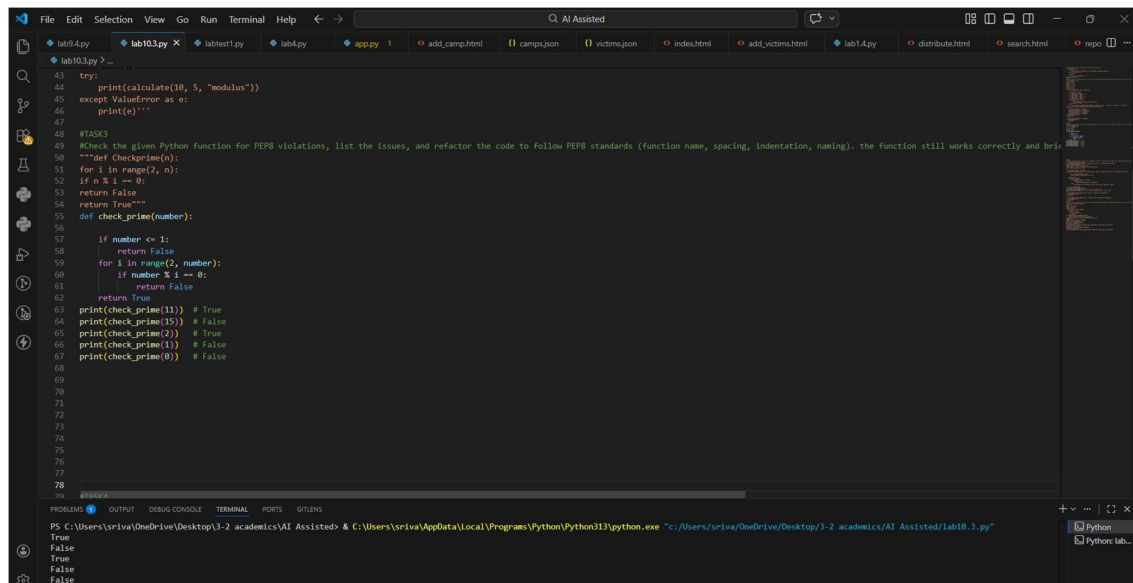
Check the given Python function for PEP8 violations, list the issues, and refactor the code to follow PEP8 standards (function name, spacing, indentation, naming). the function still works correctly .

```
"""def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True"""
```

CODE:

```
def check_prime(number):  
    if number <= 1:  
        return False  
    for i in range(2, number):  
        if number % i == 0:  
            return False  
    return True  
  
print(check_prime(11))  
print(check_prime(15))  
print(check_prime(2))  
print(check_prime(1))  
print(check_prime(0))
```

OUTPUT:



```
43 try:
44     print(calculate(10, 5, "modulus"))
45 except ValueError as e:
46     print(e)
47
48 #TASK3
49 #Check the given Python function for PEP8 violations, list the issues, and refactor the code to follow PEP8 standards (function name, spacing, indentation, naming). the function still works correctly and bri
50 """def check_prime(n):
51     for i in range(2, n):
52         if n % i == 0:
53             return False
54     return True"""
55 def check_prime(number):
56
57     if number <= 1:
58         return False
59     for i in range(2, number):
60         if number % i == 0:
61             return False
62     return True
63 print(check_prime(11)) # True
64 print(check_prime(15)) # False
65 print(check_prime(2)) # True
66 print(check_prime(4)) # False
67 print(check_prime(0)) # False
68
69
70
71
72
73
74
75
76
77
78
79
80
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS

PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted> & C:\Users\sriva\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted\lab10.3.py"

True
False
True
False
False

Analysis:

The function `check_prime(number)` checks if a number is prime. If the number is less than or equal to 1, it returns False because such numbers are not prime.

It then checks numbers from 2 to `number - 1` to see if the number is divisible. If no divisor is found, it returns True, meaning the number is prime.

The print statements test the function with different numbers.

TASK-4:

PROMPT:

Review the given Python function for readability, naming, reusability, and edge cases. Suggest improvements using better naming, type hints, input validation, and a configurable multiplier.

```
'''from typing import List, Union

def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:

    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
```

CODE:

```
from typing import List, Union
```

```
def double_even_numbers(numbers: List[Union[int, float]], multiplier: int = 2) ->
List[Union[int, float]]:
```

```
    if not isinstance(numbers, list):
```

```
        raise TypeError("Input must be a list")
```

```
    doubled_evens = []
```

```
    for num in numbers:
```

```
        if isinstance(num, (int, float)):
```

```
            if num % 2 == 0:
```

```
                doubled_evens.append(num * multiplier)
```

```
        else:
```

```
            raise TypeError("All elements in the list must be integers or floats")
```

```
    return doubled_evens
```

```
# Testing the refactored function
```

```
print(double_even_numbers([1, 2, 3, 4, 5, 6]))
```

```
print(double_even_numbers([1.5, 2.0, 3.5, 4.0], multiplier=3))
```

```
try:
```

```
    print(double_even_numbers("not a list"))
```

```
except TypeError as e:
```

```
    print(e)
```

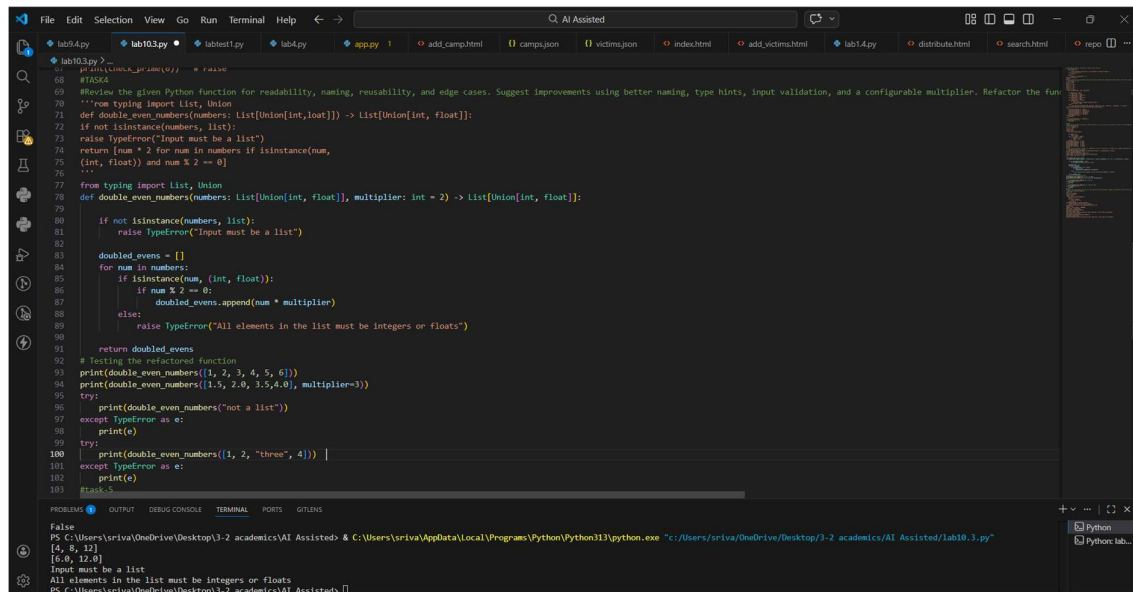
```
try:
```

```
    print(double_even_numbers([1, 2, "three", 4]))
```

```
except TypeError as e:
```

```
    print(e)
```

OUTPUT:



```
File Edit Selection View Go Run Terminal Help
lab10.3.py lab10.3.py labtest1.py lab4.py app.py add_camp.html campajon victimajon index.html add_victim.html lab14.py distribute.html search.html repo
lab10.3.py
#TASK4
69 #Review the given Python function for readability, naming, reusability, and edge cases. Suggest improvements using better naming, type hints, input validation, and a configurable multiplier. Refactor the fun
70 """from typing import List, Union
71 def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
72     if not isinstance(numbers, list):
73         raise TypeError("Input must be a list")
74     return [num * 2 for num in numbers if isinstance(num,
75             (int, float)) and num % 2 == 0]
76 ...
77 from typing import List, Union
78 def double_even_numbers(numbers: List[Union[int, float]], multiplier: int = 2) -> List[Union[int, float]]:
79
80     if not isinstance(numbers, list):
81         raise TypeError("Input must be a list")
82
83     doubled_events = []
84     for num in numbers:
85         if isinstance(num, (int, float)):
86             if num % 2 == 0:
87                 doubled_events.append(num * multiplier)
88             else:
89                 raise TypeError("All elements in the list must be integers or floats")
90
91     return doubled_events
92 # Testing the refactored function
93 print(double_even_numbers([1, 2, 3, 4, 5, 6]))
94 print(double_even_numbers([1.5, 2.0, 3.5, 4.0], multiplier=3))
95 try:
96     print(double_even_numbers("not a list"))
97 except TypeError as e:
98     print(e)
99 try:
100     print(double_even_numbers([1, 2, "three", 4]))
101 except TypeError as e:
102     print(e)
103 #Task-5
104
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS
False
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted> & C:\Users\sriva\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted\lab10.3.py"
[4, 8, 12]
[6, 9, 12, 0]
Input must be a list
All elements in the list must be integers or floats
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted>
```

Analysis:

The function takes a list of numbers and multiplies only the even numbers by a given multiplier (default = 2).

It checks input validation – the input must be a list and elements must be integers or floats, otherwise it raises a TypeError.

It returns a new list containing the multiplied even numbers, and the test cases also show error handling.

Task-5

PROMPT:

Analyze the given Python function for time complexity and performance. Suggest improvements and provide an optimized version. Compare execution time before and after optimization and briefly discuss readability vs performance.

```
"""def sum_of_squares(numbers):
```

```
total = 0
```

```
for num in numbers:
```

```
total += num ** 2
```

```
return total
```

CODE:

```
def sum_of_squares(numbers):
```


Analysis:

The code calculates the sum of squares of numbers in a list using two functions: the original loop method and an optimized method using `sum()` with a generator expression.

It creates a large list (1 to 1,000,000) and measures the execution time of both functions using the `time` module.

The optimized function is usually faster and shorter, improving performance and readability.